TSPWR

Kamil Borkowski

May 2026

1 Mathematical form of the problem

Problem Description

For our research we consider Traveling Salesman Problem With Refueling (TSPWR), which is a variant of the Traveling Salesman Problem where a vehicle must visit all cities exactly once and return to the depot, while respecting a limited fuel capacity. The vehicle consumes fuel proportional to travel distance.

Sets

We operate on a set of nodes which represent points (cities) visited.

$$V = \{0, 1, \ldots, n-1\}$$

where node 0 represents the depot.

Parameters

We consider distance and fuel capacity as parameters.

$$d_{ij} \geq 0 \quad \text{travel cost / fuel consumption from } i \text{ to } j$$

$$L \quad \text{fuel tank capacity}$$

Decision Variables

Routing variables:

$$x_{ij} \in \{0, 1\} \quad \forall i \neq j$$

$$x_{ij} = 1 \text{ if the route goes directly from } i \text{ to } j$$

Subtour elimination (MTZ variables):

$$u_i \in [0, n-1]$$

In order to prevent taking subtours under consideration, we introduce MTZ (Miller-Tucker-Zemlin). Position variable u_i is assigned to each point in the tour (city).

Fuel variables:

$$f_i \in [0, L]$$

where f_i is the remaining fuel upon arrival at node i.

Objective Function

Minimize total travel cost:

$$\min \sum_{i \in V} \sum_{j \in V, j \neq i} d_{ij} x_{ij}$$

Constraints

1. Visit each node exactly once

Each node has exactly one outgoing edge:

$$\sum_{j \in V, j \neq i} x_{ij} = 1 \quad \forall i \in V$$

Each node has exactly one incoming edge:

$$\sum_{i \in V, i \neq j} x_{ij} = 1 \quad \forall j \in V$$

2. Subtour elimination (MTZ formulation)

$$u_i - u_j + n x_{ij} \leq n - 1 \quad \forall i, j \in V \setminus \{0\}, i \neq j$$

This constraint prevents disconnected cycles that do not include the depot by ensuring that if the route goes from node i to node j, then j must appear later in the tour than i.

3. Initial fuel condition

$$f_0 = L$$

The vehicle starts with a full tank at the depot.

4. Fuel propagation constraints

If the arc (i, j) is used, fuel decreases by the travel cost:

$$f_j = f_i - d_{ij} \quad \text{if } x_{ij} = 1$$

Linearized using a big-M formulation:

$$f_j \leq f_i - d_{ij} + M(1 - x_{ij}) \quad \forall i \in V,\ \forall j \in V \setminus \{0\}$$

$$f_j \geq f_i - d_{ij} - M(1 - x_{ij}) \quad \forall i \in V,\ \forall j \in V \setminus \{0\}$$

These ensure consistency only when $x_{ij} = 1$.

5. Fuel capacity bounds

$$0 \leq f_i \leq L \quad \forall i \in V$$

The vehicle cannot exceed tank capacity or go below zero fuel.

6. Big-M constant

Big-M is a technique used in MILP to model logical conditions by introducing a large constant M. It enables constraints to be active only when a binary variable takes value 1, while becoming non-binding otherwise. In this model, it is used to enforce fuel propagation only along selected arcs.

$$M = L + \max_{i,j} d_{ij}$$

This ensures the relaxation is valid but inactive when $x_{ij} = 0$.

Summary

This model extends the classical TSP by adding a continuous resource dimension (fuel). The problem becomes a Mixed-Integer Linear Program (MILP) with routing decisions and resource feasibility constraints.

2 Metaheuristics and pseudocode

To search for solution in this constrained space, we develop a discrete Ant Colony Optimization algorithm in which artificial ants iteratively construct feasible routes using pheromone-guided probabilistic decisions and heuristic distance information.

Ant Colony Optimization for TSPWR

Require: D, L, N_a, N_{iter}, α, β, ρ, Q
Ensure: π_{best}, C_{min}

```
Initialize pheromone matrix τ
Set C_min to ∞
for iter = 1 to N_iter do
    for k = 1 to N_a do
        Create ant k
        Place ant k in depot (city 0)
        Set π = [0]
        Set current = 0
        Set fuel = L
        Set cost = 0
        Mark all cities except depot as unvisited
        while there exist unvisited cities do
            Build feasible city set F according to the fuel constraint
            if F is empty then
                Mark solution infeasible
                Break
            end if
            Select next city j probabilistically based on pheromones and heuristics
            Move to city j
            Update fuel = fuel − d_{current,j}
            Update cost = cost + d_{current,j}
            Append j to π
            Mark j as visited
            Set current = j
        end while
        Return to depot
        Update cost
        if solution is feasible then
            if cost < C_min then
                Set C_min = cost
                Set π_best = π
            end if
        end if
    end for
    Evaporate pheromone
    Deposit pheromone on π_best according to (Q/C_min)
end for
```

3 Computational complexity

3.1 Time complexity of the algorithm

The computational complexity of the proposed Ant Colony Optimization (ACO) algorithm is primarily determined by the number of ants and iterations. In each iteration, every ant constructs a feasible solution by sequentially selecting the next city, which requires evaluating up to $O(n)$ candidate transitions. Therefore, the time complexity of one iteration is:

$$O(N_a \cdot n^2)$$

where N_a denotes the number of ants and n is the number of cities.

Over N_{iter} iterations, the overall time complexity of the algorithm is

$$O(N_{iter} \cdot N_a \cdot n^2)$$

The additional cost of pheromone evaporation and updating is $O(n^2)$ per iteration, which does not change the overall asymptotic complexity.

3.2 Space complexity

In terms of space complexity, the algorithm requires storage for the distance matrix and pheromone matrix, both of size $O(n^2)$. Additionally, each ant maintains a route representation and auxiliary variables such as visited status and remaining fuel, which require linear memory $O(n)$ per ant. However, this does not dominate the overall memory usage. As a result, the total space complexity of the algorithm is $O(n^2)$.

4 Computational complexity

4.1 Experimental parameters

The following parameter values were used in the experiments:

- Map size: 200×200
- Fuel capacity: $L = 2000$ (set for a high value for the algorithm to always get a optimal solution)
- Number of ants: $N_a = 100$
- Number of iterations: $N_{iter} = 50$
- Influence of pheromone: $\alpha = 0.5$
- Influence of heuristic information: $\beta = 4.0$
- Evaporation rate: $\rho = 0.2$
- Pheromone deposit constant: $Q = 50$

For all of the shown above being constants and number of cities n differing between research samples. Optimal solution was calculated using the Gurobi optimizer.

4.2 Comparison table

Table 1: ACO Scalability vs. Gurobi (Averaged over 5 Runs)

Number of cities n	Average ACO cost	Optimal cost	Gap (%)	Average ACO time	Gurobi time
5	554.29	554.29	0.000	0.026	0.038
10	681.87	681.87	0.000	0.036	0.271
15	826.02	826.02	0.000	0.056	0.634
25	934.26	933.61	0.0696	0.162	1.478

5 Conclusion

The obtained results lead to the following conclusions:

- The ACO algorithm achieves optimal solutions for small and medium instances ($n \leq 15$), with a 0% optimality gap compared to Gurobi.

- For larger instances ($n = 25$), a small deviation from the optimal solution is observed, resulting in a gap of 0.0696%, which remains negligible in practice.
- The ACO algorithm demonstrates significantly better scalability in terms of computation time compared to the Gurobi solver.
- While Gurobi's execution time increases rapidly with problem size, ACO maintains relatively stable and low computational cost.
- Overall, ACO provides a strong trade-off between solution quality and computational efficiency, making it suitable for larger problem instances where exact methods become computationally expensive.